Documento de Arquitectura — EcoRuteando

> **Proyecto:** Software para la creación de la aplicación "EcoRuteando" **Versión:** 1.1 · 2026-09-15 · Rama `develop` · Última revisión: 2026-09-15 **Basado en:** SRS EcoRuteando, código fuente (`EcoRuteandoBackend` , `EcoRuteando/front-end`) e infraestructura (`docker-compose.yml`) **Cambios v1.1:** Alcance reducido a vehículo y a pie (ADR-8), trazabilidad RF corregida, 10 diagramas de actividad nuevos, normalización de naming.

---

1. Introducción y propósito

Este documento describe la arquitectura de software de **EcoRuteando**, portal web que sugiere rutas urbanas sostenibles en modos **vehículo y a pie (caminando)** — alcance reducido por decisión de alcance 2026-09: se descartan bus y bicicleta — con cálculo de impacto ambiental (CO_2 ahorrado) y tiempo estimado de recorrido.

Su propósito es servir de referencia técnica para desarrolladores, gestores de movilidad y evaluadores académicos, complementando el SRS y los diagramas UML (despliegue, casos de uso y actividad).

2. Alcance

- Arquitectura lógica (capas y módulos) del backend .NET.
- Arquitectura del frontend React.
- Arquitectura de datos (PostgreSQL + PostGIS, migraciones Liquibase).
- Infraestructura de despliegue (Docker Compose).
- Decisiones arquitectónicas, riesgos y conclusiones.

No cubre: diseño detallado de UI, plan de pruebas ni integraciones en tiempo real con operadores de transporte (fase futura).

3. Stack tecnológico

| Capa | Tecnología | Versión | |---|---|---| | Frontend | React + Vite + Tailwind CSS | Node 22 | | i18n | react-i18next (es/en/pt/fr) | — | | Web server / proxy | nginx (alpine) | latest | | API | ASP.NET Core (.NET) | 9.0 | | CQRS / mensajería | MediatR | 14.x | | Validación | FluentValidation | 11.x | | ORM | Entity Framework Core + Npgsql | 9.0 | | Base de datos | PostgreSQL + PostGIS | 16 / 3.5 | | Migraciones | Liquibase (+ JDBC PostgreSQL 42.7.8) | 5.0.2 | | Autenticación | JWT Bearer + BCrypt.Net | — | | Correo | MailKit → SMTP Gmail :587 | 4.x | | Logging | Serilog (console + file) | 8.x | | Documentación API | Swashbuckle (OpenAPI/Swagger) | 6.x | | Contenedores | Docker Compose | v2 |

4. Vista general de la arquitectura

Arquitectura **web de tres capas desplegada en contenedores**, orquestada por Docker Compose:

```
Navegador ──HTTP :3007──► nginx (SPA + proxy inverso)
                            │  /api/** ──HTTP──► API .NET :5124
                                                  │ Npgsql :5432
                                                  ├──► PostgreSQL 16 + PostGIS ◄── JDBC –
                                                  └──SMTP :587──► Gmail (códigos de verif
```

- El frontend es una **SPA estática** servida por nginx; no hay renderizado en servidor.
- Toda comunicación cliente-API pasa por el **proxy inverso** (`/api/**`), evitando CORS en producción.
- La API es **stateless** (JWT), lo que permite escalar réplicas horizontalmente sin afinidad de sesión.

5. Arquitectura del backend

Organización **modular por dominio de negocio** con separación interna en capas (estilo DDD ligero):

```
src/
├── Api/EcoRuteando.Api                  # Host ASP.NET Core: DI, middleware, Swagger, Seri
├── Shared/EcoRuteando.Shared            # Utilidades transversales compartidas
└── Modules/
    └── Security/
        ├── .Domain                      # Entidades y abstracciones (interfaces de reposit
        ├── .Application                 # Casos de uso: comandos/consultas (MediatR) + val
        ├── .Infrastructure              # EF Core, BCrypt, MailKit, JWT, jobs, autorizació
        ├── .Presentation                # Controladores / endpoints REST
        └── .Security                    # Composición del módulo (registro de dependencias
```

5.1 Responsabilidad de cada capa

| Proyecto | Responsabilidad | Dependencias clave | |---|---|---| | **Domain** | Entidades (`Entities`), contratos de persistencia (`Repositories`). No depende de frameworks | — | | **Application** | Casos de uso (handlers MediatR), reglas de aplicación, validación de entrada | MediatR, FluentValidation | | **Infrastructure** | Implementación técnica: EF Core/Npgsql, hash de contraseñas (BCrypt), envío de correo (MailKit + plantillas HTML), emisión/validación JWT, trabajos en segundo plano (Channels), políticas de autorización | EF Core 9, Npgsql, MailKit, BCrypt | | **Presentation** | API REST: controladores, DTOs, códigos HTTP | — | | **Api** | Punto de arranque: configuración, autenticación JwtBearer, logging Serilog, Swagger para pruebas | — |

5.2 Flujo de una petición (patrón CQRS-lite)

```
HTTP Request → Controller (Presentation) → Command/Query (Application)
            → Validator (FluentValidation) → Handler (MediatR)
            → Repository (Domain iface) → EF Core (Infrastructure) → PostgreSQL
```

La dirección de las dependencias apunta siempre hacia el **dominio**: Infrastructure implementa las interfaces definidas en Domain, manteniendo el núcleo independiente del framework (principios SOLID / Clean Architecture).

5.3 Funcionalidad implementada (módulo Security)

Registro con verificación de correo, inicio de sesión con JWT, refresh/logout, OAuth (Google/Facebook), 2FA, recuperación y restablecimiento de contraseña, gestión de sesiones, perfil de usuario y auditoría básica — correspondientes a los CU01–CU05, CU12–CU15 del SRS.

6. Arquitectura del frontend

- **SPA React + Vite**: build estático (`dist/`) servido por nginx.
- Estructura por *features*: `features/auth` , `features/home` , `features/admin` , con componentes compartidos en `shared/` .
- **Contextos globales**: tema (claro/oscuro), idioma (i18next) y autenticación (`AuthContext`).
- **Capa de servicios**: `services/authService.js` sobre un cliente axios centralizado (`api/api.js`) contra `/api` .
- **Rutas protegidas** según rol (`userRole` en localStorage): `/dashboard` , `/admin/*` , `/profile` .
- Accesibilidad y UX: modo oscuro, responsive, toasts (`react-hot-toast`), mapas interactivos con Leaflet/OpenStreetMap.

7. Arquitectura de datos

- **PostgreSQL 16 + extensión PostGIS 3.5**: soporte geoespacial nativo para rutas vehiculares/peatonales y puntos de interés (alcance actual: solo vehículo y a pie; ciclorrutas y transporte público fuera de alcance).
- **Esquema gestionado por Liquibase** (`changelog/db.changelog-master.yaml`): versiona el DDL y se ejecuta como tarea *one-shot* antes de levantar el backend (`service_completed_successfully`), garantizando que la API nunca corre contra un esquema obsoleto.
- Persistencia mediante **volumen nombrado** `postgres_data` .
- Acceso externo de administración expuesto en el puerto **5450** (solo desarrollo).

8. Despliegue e infraestructura

Servicios definidos en `docker-compose.yml` :

| Servicio | Imagen/build | Puertos | Notas | |---|---|---|---| | frontend | build multi-stage (node → nginx) | 3007→80 | proxy `/api/**` al backend | | backend | build .NET 9 | 5124→5124 | depende de postgres (healthy) y liquibase (done) | | liquibase | liquibase 5.0.2 + JDBC | — | ejecuta migraciones y termina | | postgres | postgis/postgis:16-3.5 | 5450→5432 | healthcheck `pg_isready` |

Políticas: `restart: unless-stopped` en servicios persistentes; variables sensibles vía `.env` (`env_file`). Ver detalle gráfico en `diagrama_despliegue.puml` .

9. Seguridad

1. **Contraseñas** cifradas con BCrypt (nunca en texto plano).
2. **Autenticación stateless** con JWT (access + refresh token); revocación de sesiones en BD.
3. **Comunicación cifrada**: HTTPS en tránsito; TLS hacia SMTP (:587).
4. **Validación doble**: cliente (UX) + servidor (FluentValidation) — nunca se confía solo en el front.
5. **Aislamiento de red**: los contenedores se comunican en red interna; solo nginx y puertos administrativos quedan publicados.
6. **Secretos** fuera del código (`.env` ignorado por git).

10. Decisiones arquitectónicas clave

| # | Decisión | Justificación | |---|---|---| | ADR-1 | Arquitectura modular por capas (DDD-lite) | Separar lógica de negocio de infraestructura; facilita agregar módulos futuros (Routes, Reports, Admin) replicando el patrón Security | | ADR-2 | CQRS-lite con MediatR | Casos de uso aislados y testeables; pipeline único para validación/logging | | ADR-3 | PostGIS en lugar de PostgreSQL simple | Consultas geoespaciales eficientes (distancias, cercanía) requeridas por RF10–RF15; vigente aun con alcance reducido a vehículo/a pie | | ADR-4 | Liquibase sobre migraciones EF | Migraciones versionadas e independientes del ORM; ejecución determinista en CI/CD | | ADR-5 | SPA + nginx proxy inverso | Desacopla front/back, elimina CORS, permite escalarlos por separado | | ADR-6 | JWT stateless | Escalabilidad horizontal sin sesiones de servidor; compatible con futuras apps móviles (RF móvil) | | ADR-7 | Docker Compose como orquestador | Simplicidad para fase académica/inicial; camino natural a Kubernetes si crece | | ADR-8 | Alcance de modos de movilidad reducido a vehículo y a pie (2026-09) | Se descartan bus y bicicleta para simplificar integración con APIs y cálculo CO_2; decisión reversible si se reincorpora PostGIS de transporte público |

11. Riesgos y deuda técnica

| Riesgo | Impacto | Mitigación propuesta | |---|---|---| | Un solo módulo (Security) implementado | Los RF de rutas/reportes aún sin backend | Replicar plantilla modular en Módulo Routes/Reports | | Puerto 5450 de BD publicado | Exposición de datos en producción | Restringir a VPN/red interna o eliminar en prod | | Credenciales por defecto en compose | Compromiso de la BD | Secretos gestionados (Docker secrets/vault) | | Tokens JWT en localStorage | Vulnerable a XSS si hay fallo frontal | Evaluar cookies httpOnly + CSRF | | Sin pruebas automatizadas visibles | Regresiones silenciosas | Pruebas unitarias (Application/Domain) + integración (Testcontainers) | | Dependencia de APIs externas (mapas/clima) | Indisponibilidad de funciones core | Caché de rutas y degradación elegante (ya contemplada en CU16/CU18) |

12. Conclusiones

La arquitectura de EcoRuteando cumple los objetivos definidos en el SRS y las buenas prácticas de ingeniería de software estudiadas:

1. **Separación clara de responsabilidades**: cada contenedor (nginx, API, BD, migraciones) y cada proyecto .NET tiene un propósito único, lo que facilita el mantenimiento, las pruebas y la

evolución del sistema (mantenibilidad — RNF9).

2. **Escalabilidad real**: la API stateless con JWT y la SPA estática permiten crecer en usuarios añadiendo réplicas sin cambios de diseño (RNF5, RNF11).
3. **Seguridad por diseño**: hash BCrypt, JWT con refresh y revocación, validación en ambas capas, red interna de contenedores y secretos externalizados (RNF4).
4. **Datos preparados para el dominio**: PostGIS da soporte nativo a la esencia del producto — rutas georreferenciadas y cálculo de CO_2 — con esquema versionado por Liquibase que garantiza despliegues reproducibles.
5. **Infraestructura reproducible**: `docker compose up` levanta todo el sistema con orden garantizado (healthchecks + migraciones), eliminando el clásico "en mi máquina funciona" y acelerando la incorporación de nuevos integrantes del equipo.
6. **Base extensible**: el patrón modular deja sentado el camino para incorporar los módulos de Rutas, Reportes ciudadanos y Administración sin refactorizar lo existente.

Como trabajo futuro se recomienda: implementar los módulos restantes siguiendo la plantilla de Security, endurecer secretos y exposición de puertos para un entorno productivo real, incorporar pruebas automatizadas y observabilidad (métricas/alertas — RNF12), y evaluar cookies httpOnly para los tokens.

Elaborado a partir del análisis del código fuente y la infraestructura del proyecto. Referencias cruzadas: `srs ecoRuteando.docx`, `casos-de-uso/`, `diagrama_despliegue.puml`, `diagramas-actividad/`.